```
flattenArray = (array) ->
  flatten = (acc,current) -> acc.concat current
  accumulate([], array, flatten)
flattenArray [[1,3],[2,8]]
# [1,3,2,8]
```

Function defined inside another function

To explore this, consider the following example that uses the existential operator to demonstrate which variables are defined where:

```
layerOne = ->
  first = yes
  second?          # False
  third?           # False
  layerTwo = ->
    second = yes
    first?         # True
    third?         # False
    layerThree = ->
      third = yes
      second?      # True
      first?       # True
```

The same rules apply no matter how far down you go. A lexical function scopes *all the way down*.

Argument scope

Function arguments follow the same lexical scoping rules as function variables, with the exception that arguments with the same name will shadow. Compare this example

```
jones = (x) ->
  smith = (y) ->
    x
    y
```

x argument to jones

y argument to smith

with the following:

```
jones = (x) ->
  smith = (x,y) ->
    x
    y
```

x argument to smith

y argument to smith